Reviewer Pack — Riemann Hypothesis for Random Fields Bound DPLL Tree Depth

Entry 697b15d893cd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 06:28:15 UTC

Riemann Hypothesis for Random Fields Bound DPLL Tree Depth

Entry ID: `697b15d893cd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 06:28:15 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Random Fields) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

> For every instance of a random k-CNF with n variables, the Riemann Hypothesis holds for its associated random field, and the depth of the DPLL search tree is upper-bounded by O(n log n). Specifically, E[DPLL_tree_depth(instance)] ≤ C_n * n log n, where C_n is a constant dependent on n.

Rationale (proposer's reasoning):

> Random fields provide a natural mathematical framework to study the statistical properties of boolean functions, and the Riemann Hypothesis has been used in various number-theoretic analyses. If this conjecture holds, it would suggest that the Riemann Hypothesis could be a powerful tool for understanding the complexity of satisfiability problems.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `98a60e7727374053`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a random k-CNF with n variables, if the depth of the DPLL search tree E[DPLL_tree_depth(instance)] exceeds O(n log n) by more than 3 standard deviations for at least one seed or the Spearman rank correlation coefficient between empirical and expected depths is < 0.5, then the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Riemann Hypothesis" AND "random fields" AND DPLL search tree" - "k-CNF complexity" AND "Riemann Hypothesis" AND bound" - "DPLL tree depth" AND random field AND O(n log n)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_kcnf(n, k):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(k):
            clause = random.sample(variables, random.randint(1, n))
            clauses.append(clause)
        return clauses

    def dpll_tree_depth(instance):
        # Simplified DPLL algorithm to estimate depth
        variables = set(range(1, len(instance) + 1))
        stack = []
        depth = 0

        while True:
            if not stack:
                if not variables:
                    return depth
                var = random.choice(list(variables))
                stack.append((var, True))
                stack.append((var, False))
                continue

            var, polarity = stack.pop()
            if polarity:
                variables.remove(var)
```

```
            else:
                variables.add(var)

            if not variables:
                return depth

        return depth

    n_values = [5, 10, 15, 20, 30, 40]
    total_depth = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per size
            instance = generate_random_kcnf(n, random.randint(1, n))
            depth = dpll_tree_depth(instance)
            total_depth += depth
            instances_tested += 1

    mean_depth = total_depth / instances_tested
    C_n = Fraction(mean_depth / (n * math.log(n)), 1)

    # Calculate the expected depth based on Riemann Hypothesis
    expected_depth = C_n * n * math.log(n)

    # Calculate Spearman rank correlation coefficient (simplified for demonstration)
    empirical_depths = [dpll_tree_depth(generate_random_kcnf(n, random.randint(1, n))) for n in n_
    expected_depths = [C_n * n * math.log(n) for n in n_values for _ in range(5)]

    def spearman_rank_correlation(empirical, expected):
        if len(empirical) != len(expected):
            return 0
        rank_empirical = {x: i for i, x in enumerate(sorted(empirical))}
        rank_expected = {x: i for i, x in enumerate(sorted(expected))}
        n = len(empirical)
        sum_d_squared = sum((rank_empirical[x] - rank_expected[x]) ** 2 for x in empirical)
        return 1 - (6 * sum_d_squared) / (n * (n**2 - 1))

    spearman_corr = spearman_rank_correlation(empirical_depths, expected_depths)

    conjecture_holds = spearman_corr >= 0.5
    counterexample = "" if conjecture_holds else "Spearman rank correlation < 0.5"

    return {
        "metric_name": "DPLL Tree Depth",
        "metric_value": mean_depth,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_dev} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Spearman rank correlation < 0.5' first_failing_s
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef1dca01.py", line 104, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ef1dca01.py", line 67, in run_trial
    C_n = Fraction(mean_depth / (n * math.log(n)), 1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevented it from producing data to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12498
2	preregistration	ollama_remote	glm4:latest	0	0	5912
3	novelty	ollama_remote	glm4:latest	0	0	4714
4	novelty	ollama_remote	glm4:latest	0	0	5684
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17193
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11707
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12377
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11964
9	judge	ollama_remote	glm4:latest	0	0	12779

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94828 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/697b15d893cd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/697b15d893cd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/697b15d893cd.tar.gz` (if generated)